

10. Deep Learning 3

Advanced Methods – Self-Attention and Transformers

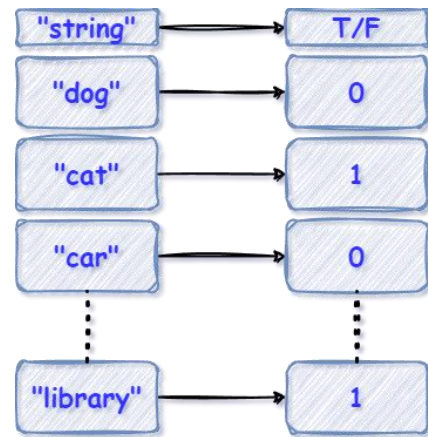
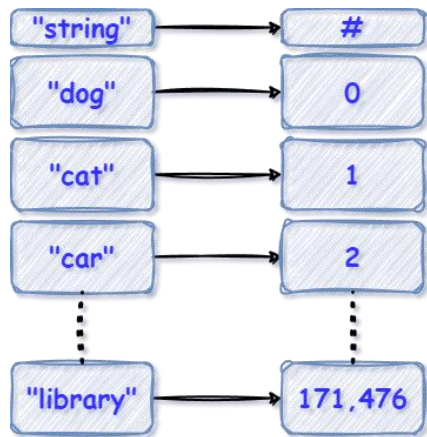
Machine learning

Jiri Chmelik

Department of Biomedical Engineering

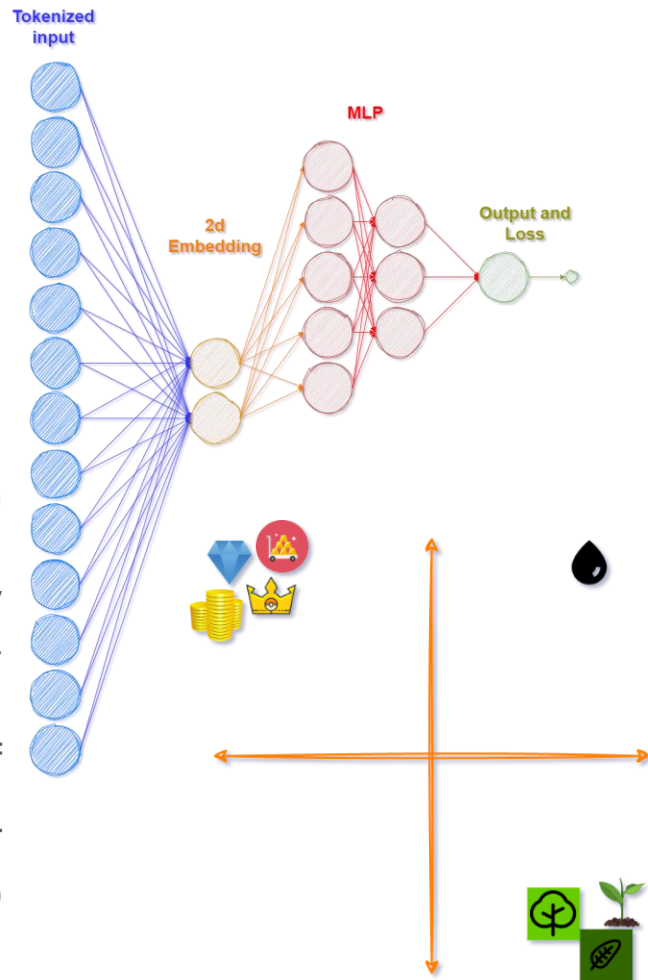
Tokenization and Embeddings

- Tokenization = conversion from original input format (words, images, etc.) into machine readable format (vector of values),
 - Could be simple vocabulary or dictionary (e. g. each unique word has unique number), sparse vector coding (for some word True value) or one-hot-encoded (for one word True value)
 - Highly ineffective (especially with large number of unique words)
 - Missing the meaning, context, similarities or relations between words



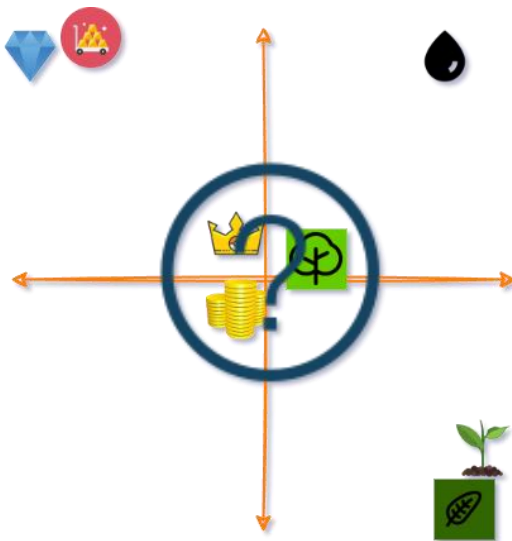
Tokenization and Embeddings

- Embedding = tokens in some d-dimensional space describing hidden relationships between tokens -> similar tokens will be close to each other in the embedding – via mapping $f: X \rightarrow Y$
 - Hundreds or thousands of dimensions (should be much lower than number of unique tokens)
 - Several approaches – most frequently using supervised learned neural networks (e. g. Word2Vec, CBOW Skip-Gram)
- Preservation of information about position of word in sentence via positional encoding – usually simple adding of vector of indices to embedding vector.



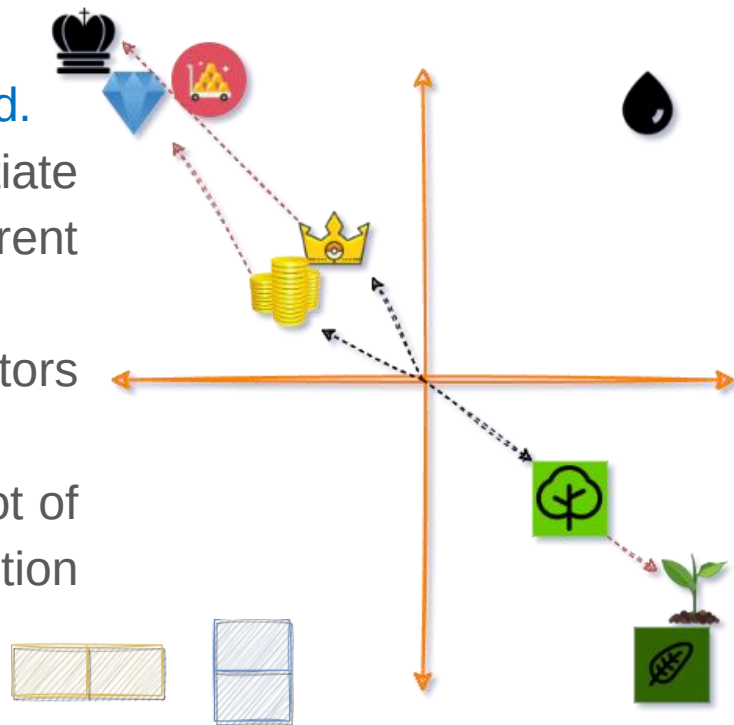
Contextual problem

- The same word could have different meaning!



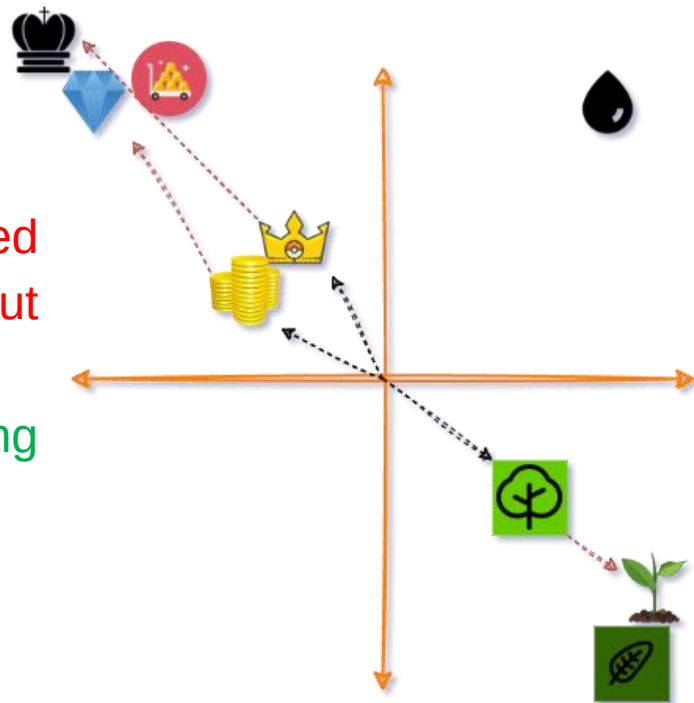
Contextual problem – solution = SELF-ATTENTION

- The **king** put on his crown.
 - A crown of the **tree** is **growing**.
 - A **valuable** Czech Crown has been invested.
 - Using contextual information to differentiate between the same words with different meaning.
1. Calculating the **similarity** between all vectors to each other as their **dot product**,
 2. Scale scores by their division by square root of vector lengths and apply SoftMax function (positive values only).
 3. Shift the positions using the similarities.

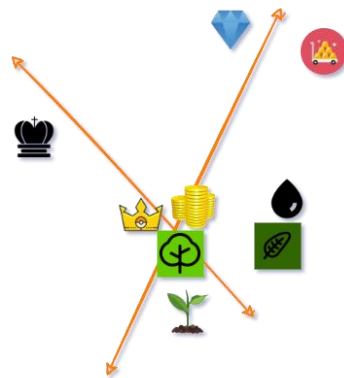


Contextual problem – solution = SELF-ATTENTION

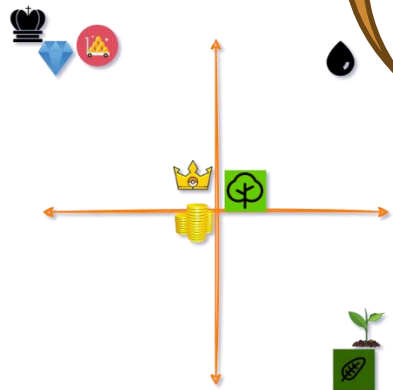
- The **king** put on his crown.
- A crown of the **tree** is **growing**.
- A **valuable** Czech Crown has been invested.
- Currency and jewellery is still not distinguished here. One embedding is not enough, But building many of them is too demanding!
- Solution: to build new embedding by modifying existing one by linear **transformation**.



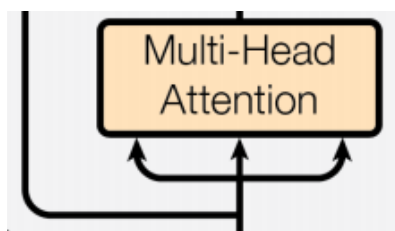
Transformers



Linear transformation of embedding
via matrix multiplication



Transformers

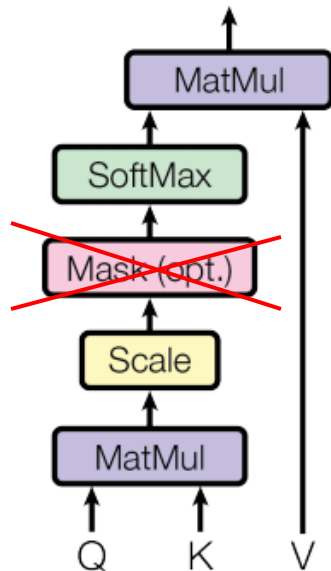


Transformers

$$MHA(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$
$$\text{head}_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention

$$A(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Q – query

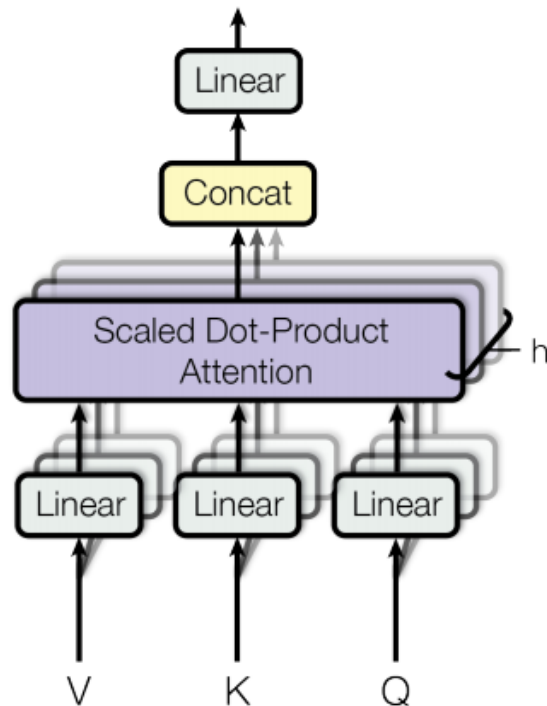
K – key

V – value

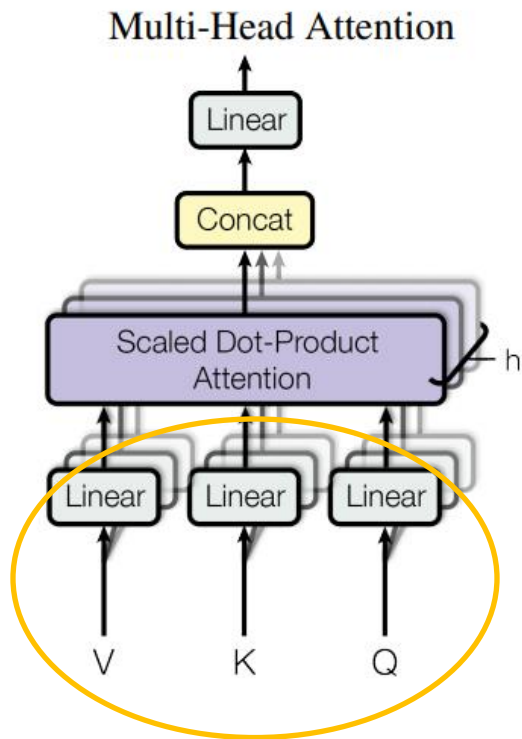
d_k – dimension of key

MatMul – Matrix implementation of Dot Product

Multi-Head Attention



Transformers



Input

Thinking

Machines

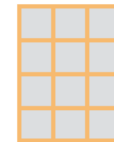
Embedding

 x_1 x_2

Queries

 q_1 q_2  W^Q

Keys

 k_1 k_2  W^K

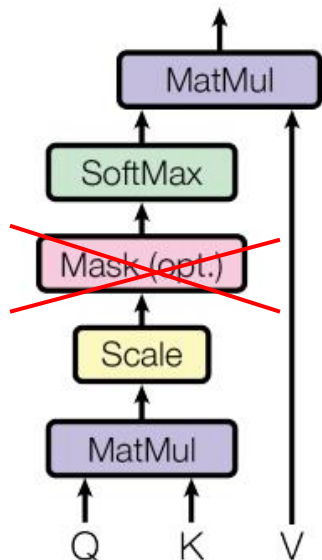
Values

 v_1 v_2  W^V

Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

Transformers

Scaled Dot-Product Attention



Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

x_1

q_1

k_1

v_1

$q_1 \cdot k_1 = 112$

14

0.88

v_1

z_1

Transformers

Machines

x_2

q_2

k_2

v_2

$q_1 \cdot k_2 = 96$

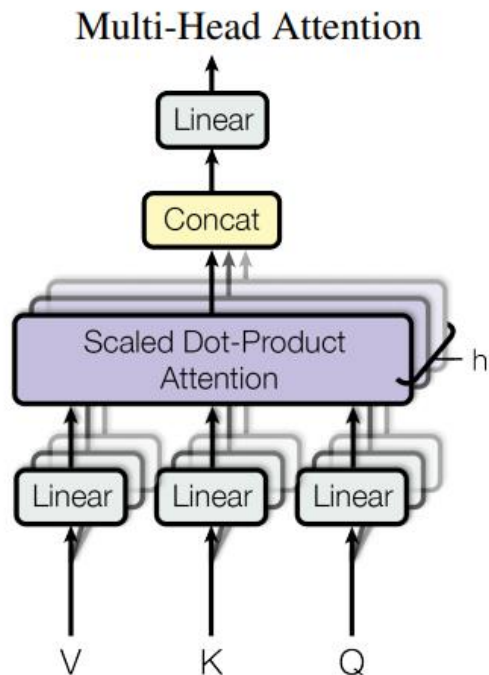
12

0.12

v_2

z_2

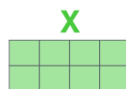
Transformers



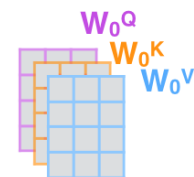
1) This is our
input sentence*

Thinking
Machines

2) We embed
each word*



3) Split into 8 heads.
We multiply X or
 R with weight matrices



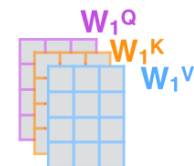
4) Calculate attention
using the resulting
 $Q/K/V$ matrices



5) Concatenate the resulting Z matrices,
then multiply with weight matrix W^O to
produce the output of the layer



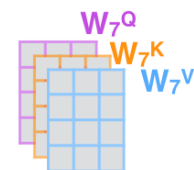
* In all encoders other than #0,
we don't need embedding.
We start directly with the output
of the encoder right below this one



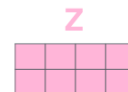
...

...

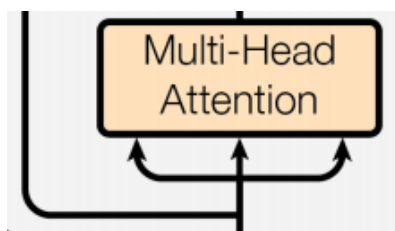
...



W^O



Transformers

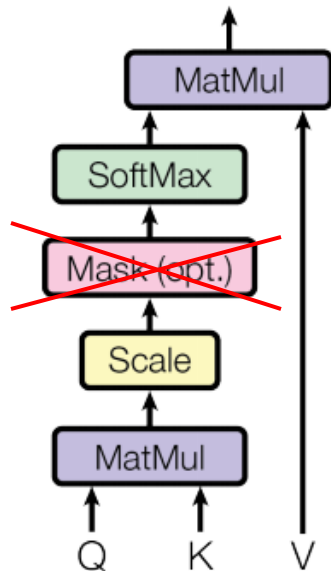


Transformers

$$MHA(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$
$$\text{head}_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention

$$A(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Q – query

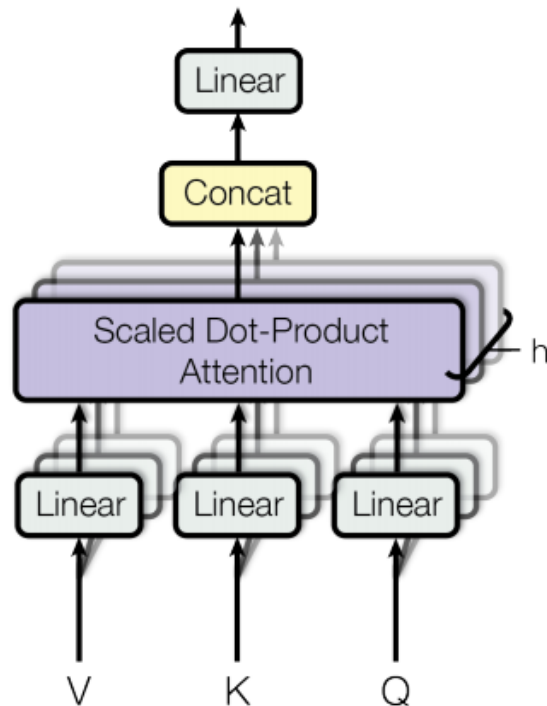
K – key

V – value

d_k – dimension of key

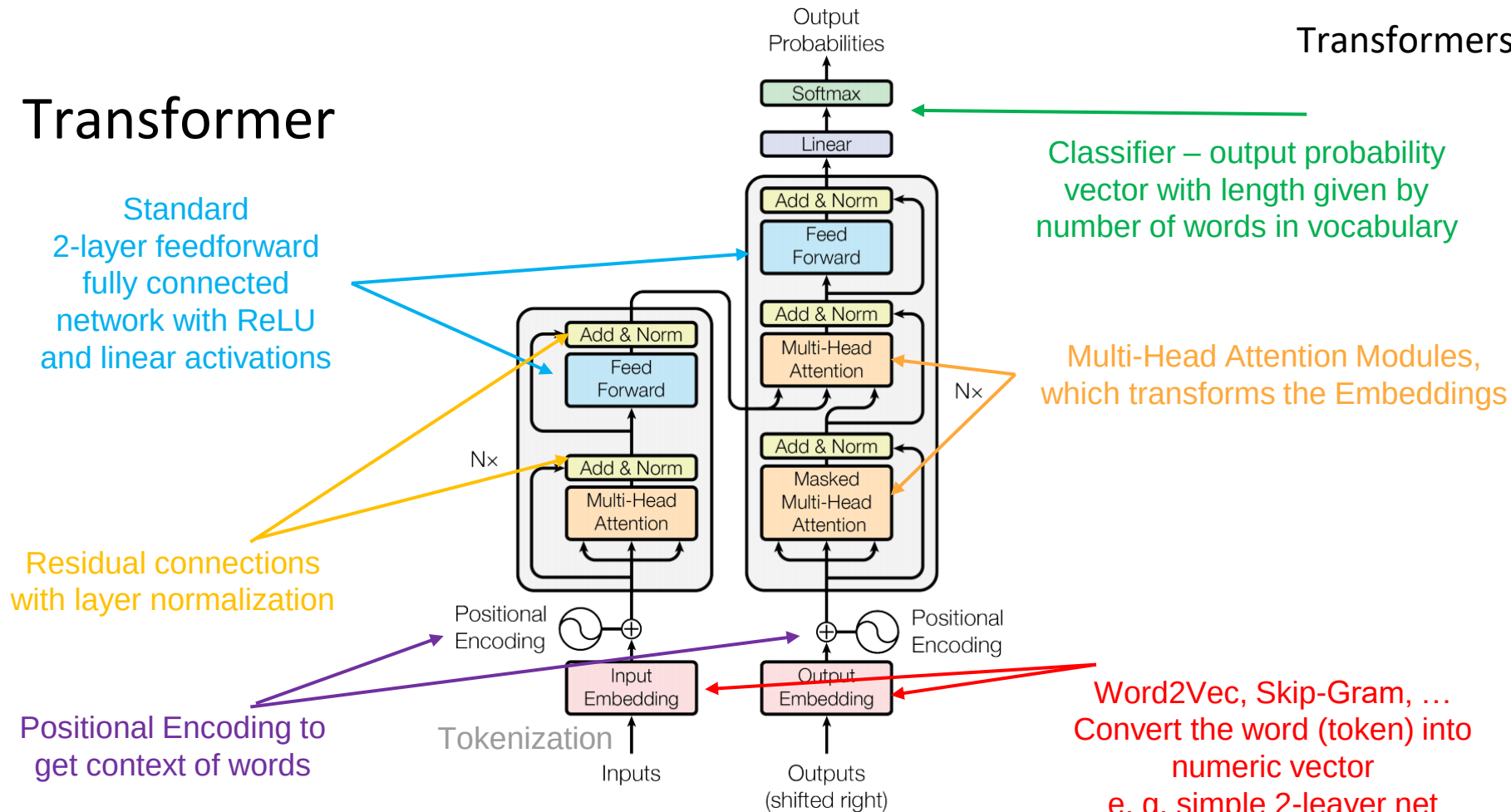
MatMul – Matrix implementation of Dot Product

Multi-Head Attention

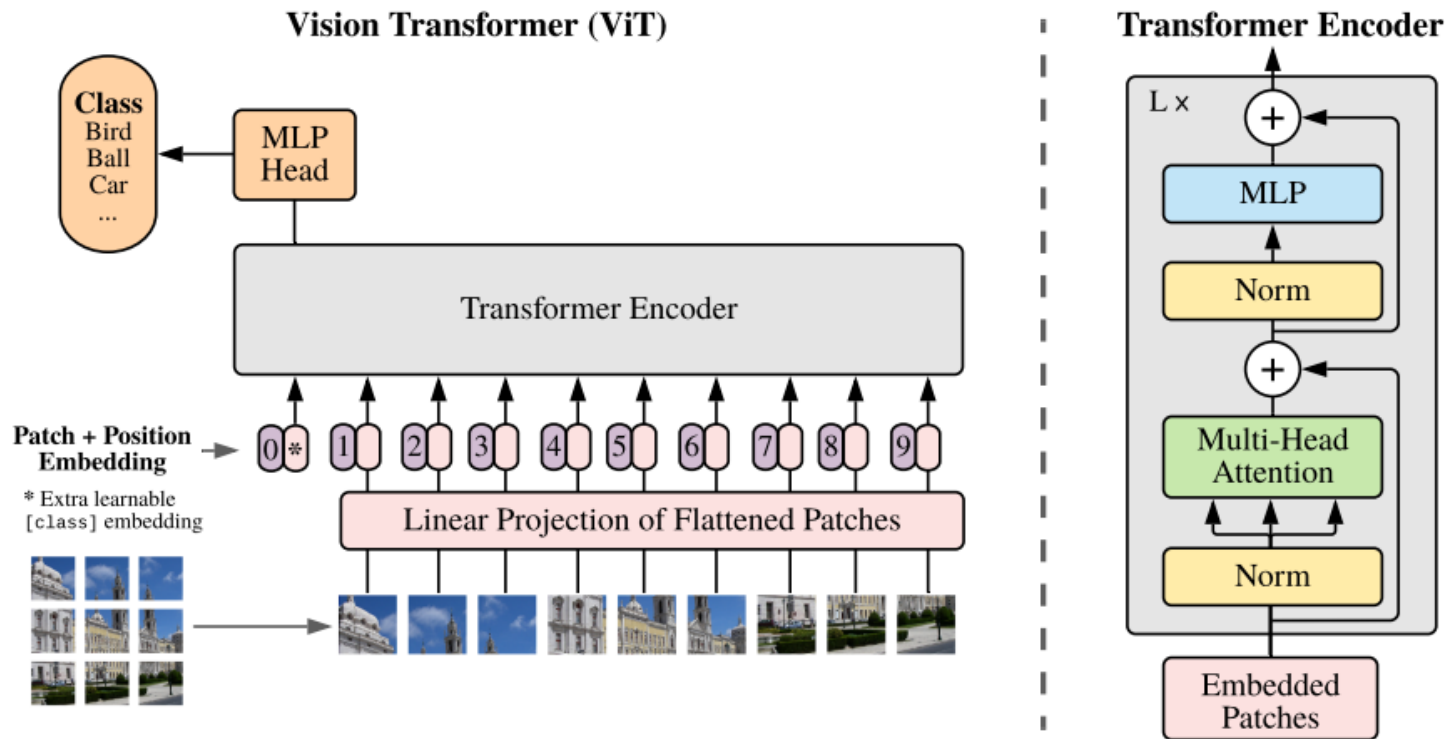


Transformer

Transformers



Vision Transformer



Transformers Summary

Properties:

- Huge computational complexity (basic model, 3.5 days on 8 Tesla P100)
- Need of “vocabulary”.
- Questions about very long or infinite signals/sentences/images/sequences.
- Real time running?
- Encoder and decoder have access to all quarry, keys and values and attention just take the important ones – replace of memory in recurrent networks.
- Can be pre-trained and fine-tuned!
- No recurrent block – parallelization is possible!

Pre-trained transformer models:

- NVIDIA Megatron-LM
- OpenAI GPT, GPT-2, GPT-3 (175 billions of params., 355 years on Tesla V100, 4.6M\$), GPT-4
- Google BERT
- Facebook RoBERTa
- Microsoft Turing-NLG
- Transformer-XL, XLNet
- etc.

Recommended sources

- [MIT: RNN, Transformers and Attention](#)
- [Serrano.Academy: Attention](#)
- [Serrano.Academy: Math behind Keys, Queries, and Values matrices](#)
- [Serrano.Academy: Transformers](#)